

HealthAI

Project Documentation

A personal health workspace — architecture, tech stack, data model, AI design, security model, and feature walkthrough of a full-stack learning project.

Repository	AI-Healthcare-Assistant (Prasoon005)
Type	Full-stack monorepo — React (web) + Express (api)
Primary Language	TypeScript
Database	PostgreSQL 16 via Prisma ORM
AI Provider	Google Gemini (gemini-2.5-flash)
Document generated	September 2026

Design principle: every number, score, or status shown in the app is either entered directly by the user or computed deterministically from data already on file. AI is used as an educational, supporting layer — never a diagnostic authority — and every hard AI safety rule is backed by a deterministic code-level check rather than trusted to a prompt alone.

Table of Contents

1. Overview
2. Design Principles
3. Core Features
4. How It Works — User Flow
5. System Architecture
6. Technology Stack
7. Data Model
8. API Surface
9. AI Capabilities & Safety Design
10. Security & Privacy
11. Project Structure
12. Getting Started
13. Environment Variables
14. Problems Solved & Engineering Challenges
15. Future Improvements

1. Overview

HealthAI is a full-stack personal health workspace built as a monorepo (**apps/web** + **apps/api**). It is explicitly **not** a diagnostic tool and does not claim to be one. It exists to keep a person's health information organized — profile, vitals, medications, documents, AI analyses, and comprehensive reports — and to help them understand that information better over time. The AI layer helps make sense of data the user already provided; it does not predict or diagnose anything.

The project was built and verified incrementally, phase by phase, against a real running database, as a full-stack learning project demonstrating production-style architecture, layered backend design, and defense-in-depth AI safety.

Why HealthAI Exists

- **One connected story:** profile, vitals, medications, documents, analyses, and reports all feed into a single searchable history and a single set of AI-generated reports, instead of living in disconnected screens.
- **No fabricated numbers:** there are no invented health scores, risk percentages, or statistics anywhere in the product — including the marketing landing page.
- **Free-tier friendly:** runs entirely on free-tier infrastructure — Google Gemini's free tier, self-hosted PostgreSQL, and Tesseract.js for local OCR — no paid APIs required.

2. Design Principles

Real data only	No fabricated health scores, risk percentages, diagnoses, or statistics anywhere in the product. Empty states say "not enough data" instead of inventing a plausible-looking placeholder.
AI as a supporting layer, not a gimmick	Gemini is used for educational analysis and report generation grounded only in data the user provided — never as a stand-in for a doctor.
Defense-in-depth safety	Every hard AI safety rule (no drug names, no diagnosis claims, escalate on emergency keywords) is backed by a deterministic code-level check. A system prompt alone is never trusted to enforce it.
Free-tier friendly	Built entirely on free-tier infrastructure so the whole system can run without any paid API subscriptions.
One connected story	Every feature writes into one unified history and analytics surface rather than being an isolated silo.

3. Core Features

Authentication	Register/login, JWT access + refresh tokens, bcrypt password hashing, "log out of all devices," and password-confirmed account deletion that cascades all owned data.
Personalized Dashboard	Wellness indicator, quick symptom check, daily planner, and a real recent-activity feed pulled from actual records.
Health Profile	The 13-field foundation (demographics, lifestyle, allergies, conditions, medications, emergency contact) that personalizes everything else in the app.
Medication Reminder	Daily dose schedule and adherence tracking.
Vitals & Trends	Heart rate, blood pressure, and SpO2 logging with trend charts.
Emergency / Medical ID	Up to 3 emergency contacts with a one-tap SOS.
Preventive Wellness	A transparent, lifestyle-based wellness indicator, explicitly labeled as a lifestyle signal, not a medical diagnosis.
Medical Vault	Upload medical documents/prescriptions with automatic OCR text extraction via Tesseract.js.
AI Health Analysis	Structured symptom analysis via Gemini, with photo upload support, deterministic emergency-keyword escalation, and drug-name redaction on every AI response.
Quick Symptom Check	A lightweight dashboard triage box that short-circuits to a full analysis recommendation for anything vague or concerning.
Comprehensive Health Reports	A multi-step AI report workflow: pick a period, answer adaptive follow-up questions, attach Medical Vault documents, get a 16-section educational report, ask follow-up questions about that report, then print/export to PDF.
Health History & Analytics	One unified, searchable, filterable timeline across every event type, plus longitudinal analytics (vital trends, medication tracking, concern-status breakdown, optional AI activity summary).
Settings	Account details, password change, "log out everywhere," full personal-data export (JSON), feedback submission, account deletion.
Landing Page	Dark/light mode (persisted, respects system preference), dynamic language selector via Intl, fully responsive layout.

4. How It Works — User Flow

The product is designed as a loop: the user builds a profile once, that profile personalizes everything downstream, and every subsequent action — a vital reading, an AI analysis, a generated report — lands in one searchable history.



5. System Architecture

The system is split into two workspace packages — a React frontend and an Express backend — connected over a versioned JSON/REST API with JWT bearer authentication and automatic refresh-on-401.

```
apps/web  (React + TypeScript + Vite)
|  axios -- JWT bearer + auto refresh-on-401
v
apps/api  (Express + TypeScript)
Route -> Controller -> Service -> Repository -> Prisma
|
+-- PostgreSQL (via Prisma ORM)
+-- Tesseract.js (OCR, local -- no external service)
+-- Google Gemini API (structured JSON output)
```

Backend Layering

Both apps follow a consistent layered pattern on the backend: **Route** → **Controller** → **Service** → **Repository** → **Prisma**. Controllers only parse requests and shape responses; services hold business logic; repositories are the only layer that touches Prisma directly. This keeps data-access, business rules, and HTTP concerns cleanly separated and independently testable.

Request Isolation & Multi-Tenancy

Every authenticated endpoint derives the acting user from the verified JWT, never from a client-supplied ID, and every database query is scoped to **userId** at the repository layer. There is no cross-user data access path anywhere in the API.

6. Technology Stack

Layer	Technologies
Frontend	React 19, TypeScript, Vite, React Router 7, Tailwind CSS 4, Recharts, Lucide Icons, Axios
Backend	Node.js, Express 5, TypeScript, Prisma ORM, Zod validation, JWT (access + refresh), bcrypt, Multer, Tesseract.js
Database	PostgreSQL 16 (Dockerized)
AI	Google Gemini (gemini-2.5-flash), structured JSON output validated against Zod schemas
Tooling	Turborepo (monorepo task orchestration), ESLint, npm workspaces
Infrastructure	Docker Compose (Postgres), local disk storage for uploads

7. Data Model

The schema is defined with Prisma and centers on the **User** model, with every other model owned by and cascade-deleted with a user. All AI-derived content (analyses, reports) is stored as structured JSON alongside the deterministic inputs that produced it.

User	Core identity: name, email, hashed password, role. Owns every other model.
HealthProfile	One-to-one 13-field profile: demographics, height/weight, smoking/alcohol, exercise/sleep, allergies, conditions, medications, emergency contact.
Medication / MedicationLog	Active medications with dosage/schedule, plus a per-dose log tracking scheduled vs. taken doses.
VitalLog	Heart rate, systolic/diastolic blood pressure, SpO2 readings with timestamps.
MedicalDocument	Uploaded file metadata (path, MIME type, size) plus OCR-extracted text.
HealthAnalysis	A symptom questionnaire submission plus the stored AI result JSON, urgency level, and a profile snapshot at the time of analysis.
Reminder	Follow-up reminders, optionally linked back to the HealthAnalysis that spawned them.
HealthReport / ReportQuestion	A generated comprehensive report: snapshots of vitals, wellness, analyses, medications, and documents at report time, the AI-generated report JSON, and a thread of report-scoped follow-up Q&A.
RefreshToken	Per-session refresh tokens, enabling individual or blanket session revocation.
Feedback	User-submitted bug reports / ideas, categorized.

All foreign keys cascade on delete from User, which is what makes account deletion a clean, single-transaction operation at the database level.

8. API Surface

The API is organized as one Express router module per resource, mounted under a common versioned prefix. Every route except authentication endpoints requires a valid JWT.

Route Module	Responsibility
auth.routes.ts	Register, login, refresh, logout, logout-all-devices
account.routes.ts	Account details, password change, account deletion, data export
user.routes.ts	Current-user profile/settings utilities
profile.routes.ts	Health Profile CRUD (13-field profile)
medication.routes.ts	Medications and medication dose logs
vital.routes.ts	Vital sign logging and trend queries
document.routes.ts	Medical Vault upload, OCR, retrieval, deletion
analysis.routes.ts	AI Health Analysis submission and retrieval
report.routes.ts	Comprehensive report generation, retrieval, and report Q&A
reminder.routes.ts	Reminders tied to analyses or created standalone
planner.routes.ts	Daily planner data for the dashboard
risk.routes.ts	Deterministic wellness/risk indicator computation
history.routes.ts	Unified searchable timeline and longitudinal analytics
feedback.routes.ts	User feedback submission

9. AI Capabilities & Safety Design

AI features are powered by **Google Gemini** (gemini-2.5-flash by default, configurable), called directly via **fetch** with a **responseSchema** so every response is structured JSON, validated against a Zod schema server-side before it is ever stored or shown to the user.

AI Features

- **Health Analysis:** takes a structured symptom questionnaire (and an optional photo) and returns an educational result — summary, considerations, self-care ideas, things to monitor, when to seek care, and questions for a doctor.
- **Quick Symptom Check:** a lighter-weight dashboard variant that recognizes common symptoms with a short note, or honestly says it needs a full analysis instead of guessing.
- **Comprehensive Reports:** a single structured call producing a 16-section report from real profile/vitals/medication/analysis/document data — never from invented information, explicitly instructed to write "Not available" rather than fabricate a section.
- **Report Q&A:** answers are grounded strictly in that report's own saved content; if the answer isn't in the report, it says so instead of guessing.
- **Longitudinal Insight:** an optional, purely descriptive summary of recorded activity, gated so it's never called when there isn't enough real activity to describe.

Deterministic Safety Backstops

The part a system prompt alone can't guarantee is enforced in code:

- A regex-based scrubber redacts roughly 50 known drug/brand names from every AI response before it's stored — added after live testing showed prompt-only instructions weren't sufficient on their own.
- An emergency-keyword scan on user input can only ever *escalate* urgency toward "seek care now" — it can never override the model's own assessment downward.
- No AI output is ever allowed to introduce a numeric health score or risk percentage; the one wellness indicator in the app is a transparent, deterministic formula over real profile fields, not an AI invention.

10. Security & Privacy

- Passwords are hashed with bcrypt; the API never returns a password hash in any response.
- JWT access tokens (15 minutes) + refresh tokens (7 days), stored per-session so any device can be revoked individually or all at once ("log out of all devices").
- Changing your password revokes every other active session automatically.
- Every resource endpoint is ownership-scoped by the authenticated user's ID from the verified JWT — never from a client-supplied ID.
- Account deletion is password-confirmed, removes every owned database row via cascading relations, and separately cleans up uploaded files from disk.
- HealthAI does **not** claim HIPAA compliance, medical certification, or any clinical partnership — none of those exist, and the product doesn't pretend otherwise anywhere in its UI or copy.

11. Project Structure

```
AI-Healthcare-System/
+-- apps/
|   +-- web/                # React frontend
|   |   +-- src/
|   |   |   +-- api/        # Typed API clients (one file per resource)
|   |   |   +-- components/ # Shared UI + landing-page components
|   |   |   +-- context/    # Auth + Theme React contexts
|   |   |   +-- hooks/      # Shared hooks (e.g. scroll-reveal)
|   |   |   +-- layouts/    # Authenticated dashboard shell
|   |   |   +-- pages/      # Route-level pages, grouped by feature
|   |   |   +-- routes/     # Router config + protected-route guard
|   |   |   +-- styles/     # Hand-rolled CSS design system
|   |   |
|   |   +-- api/            # Express backend
|   |   +-- prisma/         # schema.prisma + migrations
|   |   +-- src/
|   |   |   +-- routes/ controllers/ services/
|   |   |   +-- repositories/ validations/ middleware/
|   |   |   +-- utils/      # response helpers, JWT, hashing, AI safety nets
|   |
|   +-- docker/             # docker-compose.yml (Postgres)
|   +-- docs/screenshots/   # README screenshots, grouped by feature
|   +-- uploads/            # Local file storage for Medical Vault (gitignored)
```

12. Getting Started

Prerequisites: Node.js 18+, Docker, and an optional free Google AI Studio API key for Gemini (the app runs without it — AI features simply return "temporarily unavailable").

```
# 1. Clone
git clone https://github.com/Prasoon005/AI-Healthcare-Assistant.git
cd AI-Healthcare-System

# 2. Start PostgreSQL
cd docker
docker compose up -d
cd ..

# 3. Configure the API
cd apps/api
cp .env.example .env
# edit .env: set DATABASE_URL, add GEMINI_API_KEY if you have one
npm install
npx prisma migrate dev
npm run dev          # -> http://localhost:5000

# 4. In a second terminal, start the frontend
cd apps/web
npm install
npm run dev          # -> http://localhost:5173
```

13. Environment Variables

Set in **apps/api/.env** (see apps/api/.env.example):

Variable	Description
PORT	API server port (default 5000)
DATABASE_URL	Postgres connection string — with the bundled docker-compose.yml as-is: postgresql://postgres:postgres@localhost:5433/ai_healthcare?schema=public
JWT_ACCESS_SECRET	Secret for signing short-lived (15m) access tokens
JWT_REFRESH_SECRET	Secret for signing longer-lived (7d) refresh tokens
GEMINI_API_KEY	Google Gemini API key — leave empty to run with AI features gracefully disabled
GEMINI_MODEL	Gemini model name (default gemini-2.5-flash)

14. Problems Solved & Engineering Challenges

Preventing AI hallucination from reaching users	Structured JSON output constrained by a Gemini responseSchema, re-validated server-side with Zod before storage, plus an explicit "write 'Not available' rather than invent" instruction for report generation.
Prompt-only safety rules proved insufficient	Live testing showed the model could still slip drug names into responses despite instructions. Solved with a deterministic regex-based scrubber (~50 known drug/brand names) applied to every AI response before storage — a code-level backstop, not a prompt-level hope.
Emergency situations must never be under-triaged	An emergency-keyword scan over user input can only escalate urgency toward "seek care now," never downgrade the model's own assessment — a one-directional deterministic override.
Cross-user data leakage risk in a multi-tenant API	Every repository query is scoped to userId derived from the verified JWT, never a client-supplied ID, eliminating an entire class of IDOR-style bugs by construction.
Fabricated-looking placeholder data	Every score, percentage, or status is either user-entered or computed deterministically; empty states explicitly say "not enough data" instead of rendering a plausible-looking placeholder that could be mistaken for real information.
OCR text trustworthiness	Tesseract.js-extracted text from uploaded documents is surfaced as reviewable, editable text rather than silently trusted as ground truth elsewhere in the app.
Session security without inconveniencing users	Short-lived (15m) access tokens paired with longer-lived (7d) refresh tokens stored per-session, enabling silent refresh-on-401 on the frontend while still allowing per-device or all-device revocation, with password changes automatically revoking every other session.
Running entirely on free-tier infrastructure	Google Gemini's free tier, self-hosted PostgreSQL via Docker, and local Tesseract.js OCR together remove any hard dependency on a paid API to run or demo the full feature set.

15. Future Improvements

- Full end-to-end regression test suite across every feature (planned as its own dedicated pass, deliberately kept separate from feature work).
- Production build, hosting, and CI/CD setup (deliberately deferred — nothing has been deployed yet).
- Full UI translation behind the existing language selector (today it's a working, persisted locale *display* preference, not yet a full i18n system).
- Optional two-factor authentication.
- A richer medication lifecycle (the current model tracks active/inactive; a fuller "course completed vs. discontinued" history is only inferred today from report follow-up answers).

Built as a full-stack learning project — architecture, safety design, and every feature in this document were built and verified incrementally, phase by phase, against a real running database.